

autonomy-light 사용자 명세서

이 문서는 cocelo-autonomy-light 를 설치해서 실행하고, 제어 프로그램에서 출력 데이터를 받기 위해 필요한 정보만 정리한다. 내부 SLAM 구조나 구현 세부사항을 몰라도 사용할 수 있도록 작성했다.

1. 사전 준비

대상 Jetson에는 아래가 이미 설치되어 있다고 가정한다.

- Ubuntu/JetPack
- ROS 2 Humble: /opt/ros/humble
- 기본 명령: bash, sudo, iproute2, python3

전달받는 파일:

```
cocelo-autonomy-light_<version>_arm64.deb
```

이 .deb 에는 autonomy-light 실행 파일, custom HeightMap 메시지, Livox driver runtime, SLAM runtime, 기본 config, 문서, 예제 subscriber가 포함된다. ROS 2 Humble 자체는 포함하지 않는다.

2. 설치

```
sudo apt install ./cocelo-autonomy-light_0.1.0-1_arm64.deb
```

이미 설치된 장비에 새 .deb 를 다시 넣을 때는 재설치한다.

```
sudo apt install --reinstall ./cocelo-autonomy-light_0.1.0-1_arm64.deb
```

설치 후 주요 경로:

경로	용도
/etc/cocelo/autonomy-light/autonomy_light.yaml	현장 설정 파일
/usr/bin/autonomy-light	실행 명령
/usr/bin/autonomy-light-doctor	설치 및 ROS topic 점검
/usr/bin/autonomy-light-heightmap-example	HeightMap 수신 예제
/opt/cocelo/autonomy-light/install	패키지 runtime install tree
/usr/share/doc/cocelo-autonomy-light	문서와 예제 코드

3. 설정

실행 전 설정 파일을 확인한다.

```
sudo nano /etc/cocelo/autonomy-light/autonomy_light.yaml
```

현장에서 주로 바꾸는 값:

항목	기본값	설명
livox_interface	enP8p1s0	MID360이 연결된 Jetson 유선 NIC
livox_host_ip	192.168.1.50/24	Jetson의 LiDAR 통신용 고정 IP
livox_lidar_ip	192.168.1.166	MID360 장치 IP
internal_ros_domain_id	42	LiDAR/SLAM 내부 ROS 2 domain
external_ros_domain_id	0	제어 프로그램이 받을 ROS 2 domain
debug_local_map_ros_domain_id	42	debug local map 출력 domain
target_frame	base_link	제어 기준 frame
lidar_frame	lidar_link	LiDAR frame
elevation_resolution	0.05	height map cell 크기, meter
elevation_x_length	1.2	height map x 길이, meter

elevation_y_length	1.2	height map y 길이, meter
publish_rate_hz	50.0	외부 출력 주기

권장 domain 구성:

```
internal_ros_domain_id: 42
external_ros_domain_id: 0
debug_local_map_ros_domain_id: 42
```

/point_lio/local_map, /livox/lidar, /livox/imu 같은 큰 데이터는 내부 domain에만 남기는 것을 권장한다. 이 topic들이 제어용 ROS domain에 보이면 네트워크와 DDS buffer에 부담을 줄 수 있다.

4. 실행

```
autonomy-light --real
```

정상 실행 시 로그에 아래와 비슷한 내용이 나온다.

```
ROS_DOMAIN_ID internal=42
ROS domains: internal=42 external=0 debug_local_map=42 (not republished)
Autonomy light ready: lidar=Lidar_Link target=base_link ...
```

설치/출력 확인:

```
autonomy-light-doctor
```

HeightMap 수신 예제:

```
ROS_DOMAIN_ID=0 autonomy-light-heightmap-example
```

5. 제어 프로그램이 받는 출력

아래 topic은 external_ros_domain_id 에서 발행된다. 기본값은 ROS_DOMAIN_ID=0 이다.

Topic	Type	QoS	설명
/autonomy_light/height_map_data	autonomy_light/msg/HeightMap	Reliable, Volatile, KeepLast depth 2	제어용 height map
/autonomy_light/height_map	sensor_msgs/msg/PointCloud2	Reliable, Volatile, KeepLast depth 2	RViz/debug용 height map point cloud
/autonomy_light/odom	nav_msgs/msg/Odometry	Reliable, Volatile, KeepLast depth 10	로봇 odometry
/path	nav_msgs/msg/Path	Reliable, Volatile, KeepLast depth 10	로봇 path
/tf	tf2_msgs/msg/TFMessage	ROS 2 TF QoS	동적 TF
/tf_static	tf2_msgs/msg/TFMessage	ROS 2 static TF QoS	정적 TF

수신 측 ROS 환경:

```
source /opt/ros/humble/setup.bash
source /opt/cocelo/autonomy-light/install/setup.bash
export ROS_DOMAIN_ID=0
```

수신 측에서 custom message 확인:

```
ros2 interface show autonomy_light/msg/HeightMap
```

6. HeightMap 메시지

메시지 타입:

```
float32[] data
float32 resolution
float32 x_length
float32 y_length
```

.deb 를 설치한 수신 장비에서는 아래처럼 packaged install tree를 source하면 custom message를 바로 사용할 수 있다.

```
source /opt/ros/humble/setup.bash
source /opt/cocelo/autonomy-light/install/setup.bash
ros2 interface show autonomy_light/msg/HeightMap
```

.deb 를 설치하지 않고 수신 프로그램 쪽에서 message interface만 직접 만들 경우에는 ROS 2 package 이름, message 이름, 필드 타입과 순서를 모두 동일하게 맞춘다.

```
package name: autonomy_light
```

```
message file: msg/HeightMap.msg
message type: autonomy_light/msg/HeightMap
```

msg/HeightMap.msg :

```
float32[] data
float32 resolution
float32 x_length
float32 y_length
```

최소 package.xml 의존성:

```
<buildtool_depend>ament_cmake</buildtool_depend>
<build_depend>roslaunch</build_depend>
<exec_depend>roslaunch</exec_depend>
<member_of_group>roslaunch</member_of_group>
```

최소 CMakeLists.txt :

```
cmake_minimum_required(VERSION 3.16)
project(autonomy_light)

find_package(ament_cmake REQUIRED)
find_package(roslaunch REQUIRED)

roslaunch_generate_interfaces(${PROJECT_NAME}
  "msg/HeightMap.msg"
)

ament_export_dependencies(roslaunch)
ament_package()
```

빌드와 확인:

```
source /opt/ros/humble/setup.bash
colcon build --packages-select autonomy_light
source install/setup.bash
ros2 interface show autonomy_light/msg/HeightMap
```

shape 계산:

```
width = ceil(x_length / resolution)
height = ceil(y_length / resolution)
data.size() == width * height
```

기본 설정에서는:

```
resolution = 0.05
x_length   = 1.2
y_length   = 1.2
width      = 24
height     = 24
data.size  = 576
```

배열 순서:

```
index = row * width + col
row: y_min -> y_max
col: x_min -> x_max
```

cell 중심 좌표:

```
x_min = -0.5 * x_length
y_min = -0.5 * y_length

x = x_min + (col + 0.5) * resolution
y = y_min + (row + 0.5) * resolution
```

값 의미:

```
data[index] = base_height - obstacle_height
base_height = algorithm.clipping.max_z
```

기본 base_height 는 0.48 이다.

```
평지:      data ~= 0.48
낮은 장애물: data < 0.48
높은 장애물: data -> 0.0
```

즉 data 는 “위에서 아래로 켜 거리값”처럼 해석하면 된다. 기존 Isaac height scanner 또는 sim-to-real pipeline에서 distance height map을 쓰는 경우 이 값을 그대로 연결하기 쉽다.

7. Python 수신 예제

패키지 설치 후 바로 실행:

```
ROS_DOMAIN_ID=0 autonomy-light-heightmap-example
```

직접 코드에서 사용할 때의 QoS:

```
from rclpy.qos import QoSProfile, ReliabilityPolicy, DurabilityPolicy, HistoryPolicy

qos = QoSProfile(
    history=HistoryPolicy.KEEP_LAST,
    depth=2,
    reliability=ReliabilityPolicy.RELIABLE,
    durability=DurabilityPolicy.VOLATILE,
)
```

subscribe topic:

```
from autonomy_light.msg import HeightMap

node.create_subscription(
    HeightMap,
    "/autonomy_light/height_map_data",
    callback,
    qos,
)
```

8. 빠른 점검 명령

외부 출력 topic 확인:

```
ROS_DOMAIN_ID=0 ros2 topic list | grep autonomy_light
ROS_DOMAIN_ID=0 ros2 topic hz /autonomy_light/height_map_data
ROS_DOMAIN_ID=0 ros2 topic echo /autonomy_light/height_map_data --once
```

odometry 확인:

```
ROS_DOMAIN_ID=0 ros2 topic hz /autonomy_light/odom
```

local map이 제어 domain에 새는지 확인:

```
ROS_DOMAIN_ID=0 ros2 topic list | grep point_lio/local_map ## 안나와야 정상
```

정상 운용에서는 아무것도 나오지 않는 것이 좋다.

내부 debug local map을 보고 싶을 때:

```
ROS_DOMAIN_ID=42 rviz2
```

RViz에서 /point_lio/local_map 이 안 보이면 PointCloud2 display의 Reliability를 Best Effort 로 바꾼다.

9. 자주 나는 문제

`autonomy_light/msg/HeightMap` 타입을 모른다고 나올 때

수신 환경에서 runtime setup을 source한다.

```
source /opt/ros/humble/setup.bash
source /opt/cocelo/autonomy-light/install/setup.bash
ROS_DOMAIN_ID=0 ros2 interface show autonomy_light/msg/HeightMap
```

topic이 안 보일 때

수신 측 ROS_DOMAIN_ID 가 external_ros_domain_id 와 같은지 확인한다.

```
grep external_ros_domain_id /etc/cocelo/autonomy-light/autonomy_light.yaml
echo $ROS_DOMAIN_ID
```

topic은 보이는데 ros2 topic echo 가 아무것도 안 찍힐 때

실행 터미널에서 `sudo autonomy-light --real` 로 실행했는지 확인한다. root로 실행된 publisher와 일반 사용자 subscriber 사이에서 DDS shared-memory 권한 문제로 discovery만 되고 데이터가 안 올 수 있다.

확인용으로 subscriber도 root로 실행해 본다.

```
sudo bash -lc '
source /opt/ros/humble/setup.bash
source /opt/cocelo/autonomy-light/install/setup.bash
```

```
ROS_DOMAIN_ID=0 ros2 topic echo /autonomy_light/odom --once
```

이 명령에서는 값이 나오고 일반 사용자 터미널에서는 안 나오면 권한 문제다. 기존 프로세스를 종료한 뒤 일반 사용자로 실행한다.

```
sudo kill -f autonomy-light || true
sudo kill -f autonomy_light || true
autonomy-light --real
```

root subscriber에서도 값이 안 나오면 내부 SLAM 입력을 확인한다.

```
ROS_DOMAIN_ID=42 ros2 topic hz /aft_mapped_to_init
ROS_DOMAIN_ID=42 ros2 topic hz /point_lio/local_map
```

MID360 연결이 안 될 때

설정 파일의 NIC와 IP를 확인한다.

```
grep -E 'livox_interface|livox_host_ip|livox_lidar_ip' /etc/cocelo/autonomy-light/autonomy_light.yaml
ip -br addr
```

RViz에서 point cloud가 안 보일 때

debug local map은 Best Effort QoS일 수 있다. RViz PointCloud2 display에서 Reliability를 Best Effort 로 설정한다.

10. 제거

```
sudo apt remove cocelo-autonomy-light
```

설정 파일까지 제거하려면:

```
sudo apt purge cocelo-autonomy-light
```